

Benchmarking and Deployment of Large Language Model Inference Frameworks in Cloud Environments

Ankit Amin, Syed Abdullah Rizvi, Arnav Dadarya, Jesse Chen, Kevin Lin, Abhiram Kidambi
University of Maryland Email: aamin18, srizvi17, arnav1, jesse, klin0513, akidamb1@terpmail.umd.edu

Abstract—Large Language Models (LLMs) have become foundational to modern artificial intelligence systems, with widespread adoption across research and industry. However, running these models at scale remains a significant challenge due to high memory constraints, token-by-token generation, latency constraints, and the weight and computation required to get meaningful output. This paper presents an empirical comparison of two widely used open-source LLM inference frameworks, vLLM and SGLang (and some associated modifications), focusing on performance tradeoffs in different cloud-based GPU environments. We benchmarked throughput, latency, and time-to-first-token across different models, batch sizes, and hardware configurations on Microsoft Azure. We conclude that while there is no “perfect” framework, the choice depends on the specific use case: we recommend SGLang for services with low concurrent users where low initial response time and CUDA graph optimizations are a priority, and vLLM for production-scale environments with high traffic because of superior throughput and lower latency when handling multiple simultaneous requests.

I. INTRODUCTION

Large Language Models such as GPT, LLaMA, and Mistral dominate contemporary AI workloads, particularly in real-time, user-facing applications with chat systems. Unlike training, inference must operate on-demand and token-by-token, leading to stringent latency and scalability requirements. As model sizes reach tens or hundreds of billions of parameters, naive inference pipelines struggle to efficiently utilize GPU memory and compute resources, while not being able to triage and balance performance with resources.

To address these challenges, several specialized inference frameworks have emerged. Among the most prominent are vLLM and SGLang, both released as open-source projects following their respective research publications in late 2023 and mid-2024, respectively. These systems propose different approaches to key problems such as KV-cache management, request scheduling, and batching. Understanding the empirical performance tradeoffs between these frameworks is critical for practitioners deploying LLMs in production environments, with many of these different choices that could impact execution.

This work benchmarks vLLM and SGLang under controlled cloud settings to evaluate their relative performance across a range of realistic inference scenarios.

II. BACKGROUND AND RELATED WORK

Both vLLM and SGLang were introduced by research papers that focused on speeding and optimizing LLMs performance. The main improvement across both LLM frameworks involves KV-cache, a key-value pairing storing previous transformer attention layers and associated computations of attention for tokens so that the model can efficiently reuse them when generating longer sequences instead of recomputing.

vLLM introduces *PagedAttention*, a memory management technique to store KV-Cache into fixed-size pages inspired by page-tables from virtual memory systems. By organizing the KV-cache into discrete KV-blocks managed by a central Block Table, vLLM effectively eliminates the memory fragmentation and “over-provisioning” that typically plagues GPU memory with long streams of KV-Cache accumulated through several prompts. Furthermore, vLLM maximizes hardware throughput through Continuous Batching, which dynamically slots in new requests as others finish to avoid idling the GPU between batch cycles.

SGLang introduces *RadixAttention*, which automatically manages the KV cache using a radix tree structure to identify and reuse shared prompt prefixes across multiple generation calls. This architecture is particularly beneficial for tasks with significant prefix overlap, and leads to significant improvement in utilization of KV-Cache for faster and more efficient computation. Additionally, SGLang utilizes a Zero-Overhead Batch Scheduler that overlaps CPU scheduling with GPU computations ensuring that GPU remains nearly 100% utilized. The cache-aware scheduling policy prioritizes requests whose states are already present in memory, which minimizes redundant processing and latency.

While the initial surge of research gave birth to these open-source projects, both frameworks have evolved significantly through a process of continuous cross-pollination. Although vLLM preceded SGLang, the two have since integrated each other’s core optimizations to push the boundaries of LLM inference speed. For instance, while SGLang pioneered Radix-Attention to facilitate KV-cache reuse for shared prefixes, vLLM subsequently implemented Automatic Prefix Caching to address the same bottleneck using a similar logic. Our research utilizes the most current iterations of these engines, both of which now incorporate reciprocal enhancements.

III. METHODOLOGY

A. Experimental Roles

For specific roles, we divided the team and had different members work on different tasks. Here is a quick overview:

- **Abhiram** - worked on testing and benchmarking vLLM using the GPT-OSS-20B model on A100s, also got benchmarks for comparing vLLM Triton and vLLM Flash-Attention by using different prompt lengths using the TinyLlama model, and contributed to presentation and poster and handled most communications

B. Benchmarking Implementation

To systematically evaluate the performance of each inference framework, we developed a Python-based benchmarking script that measures key metrics across varying batch sizes. The script interfaces with SGLang’s OpenAI-compatible API endpoint and collects fine-grained performance data for each request.

1) *Test Configuration*: The benchmarking script was configured with the following parameters:

- **Model**: TinyLlama-1.1B-Chat-v1.0, gpt-oss-20B
- **Batch Sizes**: [1, 2, 4, 8, 16, 32, 64, 128]
- **Number of Prompts**: 64 unique prompts per test
- **Maximum Tokens**: 64 tokens per completion
- **Endpoint**: Local SGLang server at `http://localhost:8000/v1/completions`

2) *Dataset*: Rather than using synthetic or manually crafted prompts, we utilized the Alpaca instruction-tuning dataset from HuggingFace (tatsu-lab/alpaca) to ensure realistic workload characteristics. The script extracts prompts by concatenating the instruction field with the optional input field, providing diverse query types representative of actual LLM usage patterns.

3) *Metric Collection*: For each request, the script captures three primary performance indicators:

Time-to-First-Token (TTFT): Measured by recording the timestamp when the HTTP POST request is initiated and comparing it to when the first token arrives via the streaming response. This metric captures the initial responsiveness of the inference engine, which is critical for user-perceived latency in interactive applications.

Per-Request Latency: Calculated as the total elapsed time from request submission to completion, including all token generation. This end-to-end measurement reflects the complete user experience for a single prompt.

Throughput: Computed at the batch level by dividing the total number of completion tokens generated across all requests by the wall-clock time required to process the entire batch. This metric evaluates the system’s ability to maximize GPU utilization under concurrent load.

4) *Concurrent Request Handling*: To simulate realistic concurrent workloads and leverage each framework’s dynamic batching capabilities, the script employs Python’s `ThreadPoolExecutor` to dispatch multiple requests in parallel. The number of concurrent threads matches the target

batch size, allowing the server to naturally batch requests together. The script uses the `as_completed()` iterator to collect results as they finish, ensuring accurate per-request timing even when requests complete out of order.

5) *Streaming Protocol*: All requests utilize the streaming completion API, which returns tokens incrementally via Server-Sent Events (SSE). The script parses each `data:` line, extracts the JSON payload, and increments the token count until the `[DONE]` sentinel or a `finish_reason` of "stop" is received. This approach mirrors real-world deployment patterns where streaming responses improve perceived latency in user interfaces.

6) *GPU Memory Monitoring*: To assess memory efficiency, the script invokes `nvidia-smi` before and after each batch to capture GPU memory utilization. While not a precise per-request metric due to asynchronous GPU operations, this provides insight into memory scaling behavior as batch size increases.

7) *Statistical Analysis*: For each batch size, the script computes aggregate statistics including mean latency, median latency (P50), and 90th percentile latency (P90). The P90 metric is particularly valuable for understanding tail latency, which often dominates user experience in production systems. All metrics are exported to CSV format for subsequent analysis and visualization.

8) *Framework-Specific Testing*: While the presented script targets SGLang, equivalent benchmarking procedures were developed for vLLM with both Triton and Flash attention backends. Each framework was tested independently on identical hardware with matching prompt sets to ensure fair comparison. The modular design of the benchmarking script allowed straightforward adaptation to different API endpoints and response formats.

C. Cloud Infrastructure

Due to the computational demands of LLM inference, experiments were conducted on cloud-hosted virtual machines. Microsoft Azure was selected for access to high-end NVIDIA GPUs. Initial plans to deploy a shared request-serving cluster were abandoned due to cost and complexity, and experiments were instead conducted on single-VM deployments.

D. Hardware

We targeted NVIDIA T4 and A100 GPUs to capture performance differences between cost-efficient and high-end hardware. While vLLM ran successfully on both platforms, recent versions of SGLang exhibited incompatibilities with T4 GPUs, preventing meaningful comparisons at that tier. Limited exploratory access was also granted to the University of Maryland’s Zaratan cluster, which includes NVIDIA H100 GPUs; however, scheduling constraints limited data collection.

E. Metrics

We evaluated each framework using the following metrics:

- **Throughput**: tokens generated per second
- **Latency**: end-to-end request completion time

Throughput comparison (tokens/sec)

Batch	SGLang + CUDA graphs	vLLM Triton	vLLM Flash
1	25.8	115.3	177.1
2	53.7	126.8	149.3
4	275.9	192.9	254.2
8	1434.1	1176.7	868.6
16	2189.5	3101.8	2531.8
32	4980.8	4952.1	5082.5
64	5466.5	8610.5	7765.3

Fig. 1. Table showing throughput in tokens/sec for different models

Time to first token (TTFT)

Batch	SGLang + CUDA graphs	vLLM Triton	vLLM Flash
1	0.038	0.365	0.156
2	0.036	0.015	0.012
4	0.069	0.017	0.012
8	0.048	0.025	0.016
16	0.046	0.024	0.019
32	0.056	0.023	0.023
64	0.142	0.027	0.027

Fig. 2. Table showing time to first token for different models

- **Time-to-First-Token (TTFT):** delay before the first output token

Experiments varied batch size, model choice, and request structure to approximate realistic deployment conditions.

IV. RESULTS

For results, we present 6 different figures which showcase our three main comparisons for each LLM and their associated performance: TTFT (Time-to-First-Token), Latency, and Throughput. Latency and Throughput measure speed in different ways, with latency being used to measure the time taken for a single task/prompt processed through the pipeline while throughput measuring all tasks performed during the pipeline taking advantage of operational efficiencies exploited by simultaneously running two stages of the LLM pipeline at the same time.

Figure 1 and Figure 4 are associated with each other, representing graph and table formats of throughput comparison in different batches of the three tested models, SGLang + CUDA, vLLM Triton, and vLLM Flash. Similarly, Figures 2 and 5 are associated with each other, representing Time-To-First-Token in the same way. Finally, Figures 3 and 6 are also associated with representing. Figures 7, 8, and 9 are all measuring the relationship between the same parameters ran on the GPT-OSS-20B (medium-size open-source model simulating some higher-intense workloads). Figures 9 and 10 refer to our research on prompt-sizes using vLLM Flash and vLLM Triton.

Average latency per request

Batch	SGLang + CUDA graphs	vLLM Triton	vLLM Flash
1	0.038	0.554	0.361
2	0.036	0.015	0.012
4	0.126	0.017	0.013
8	0.144	0.120	0.094
16	0.120	0.179	0.159
32	0.156	0.163	0.192
64	0.316	0.214	0.208

Fig. 3. Table showing average latency per request for different models

Batch Size vs Throughput (tok/s) on A100

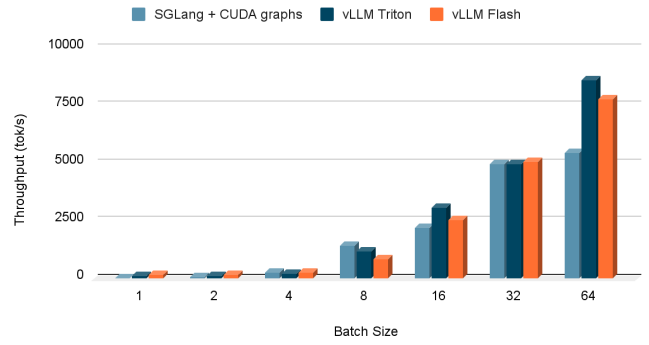


Fig. 4. Graph showing throughput for different batch sizes on A100

V. DISCUSSION

The comparative analysis between SGLang with CUDA graph optimizations and vLLM with Triton and Flash kernels reveals distinct architectural advantages depending on the operational scale. The most striking difference observed in the benchmarks is the performance of SGLang with CUDA graphs during the initial request phase. By recording a sequence of CUDA operations and replaying them as a single execution unit on the GPU, SGLang effectively eliminates communi-

Batch Size vs Avg Latency (s) on A100

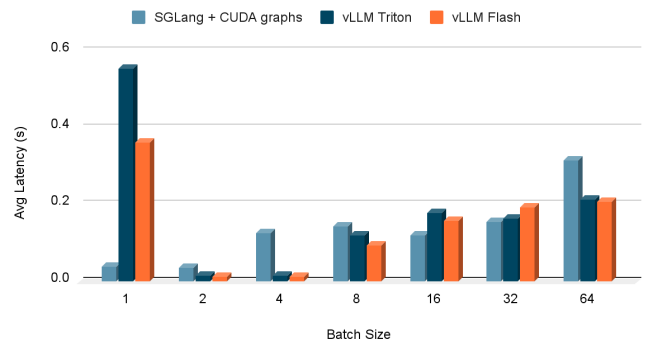


Fig. 5. Graph showing latency for different batch sizes on A100

Batch Size vs Avg TTFT (s) on A100

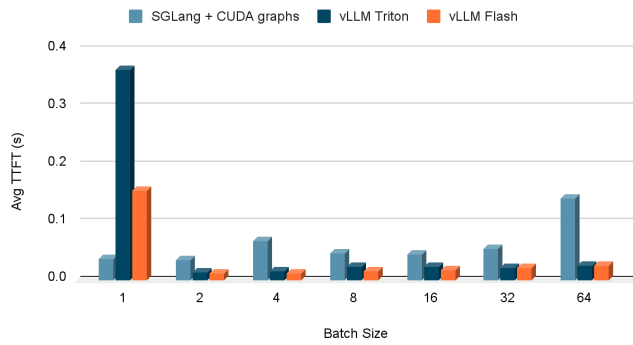


Fig. 6. Graph showing Time-to-First-Token for different batch sizes

Batch Size vs Avg Latency (s) on A100 on GPT-OSS-20B

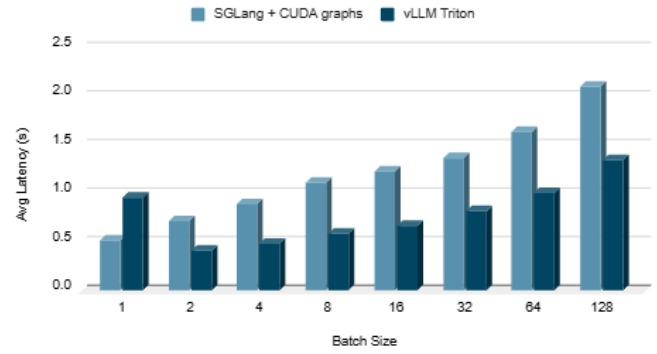


Fig. 9. Batchsize vs Average Latency on A100 on GPT-OSS-20B

Batch Size vs Throughput (tok/s) on A100 on GPT-OSS-20B

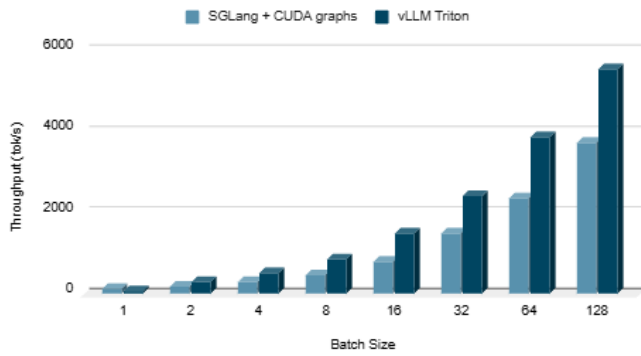


Fig. 7. Batchsize vs Throughput on A100 on GPT-OSS-20B

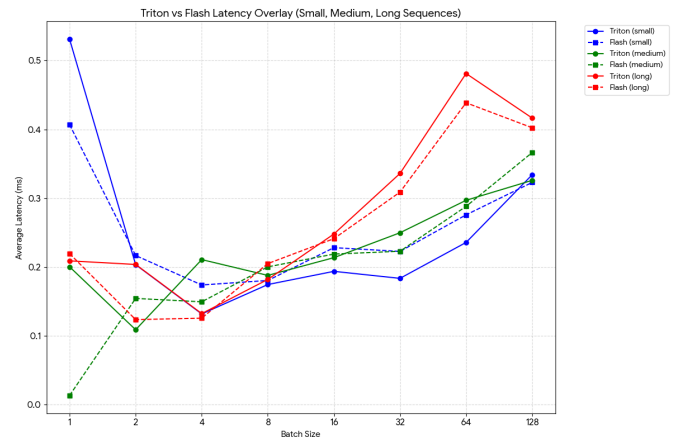


Fig. 10. Average TTFT vs Batchsize for Larger Prompts

Batch Size vs Avg TTFT (s) on A100 on GPT-OSS-20B

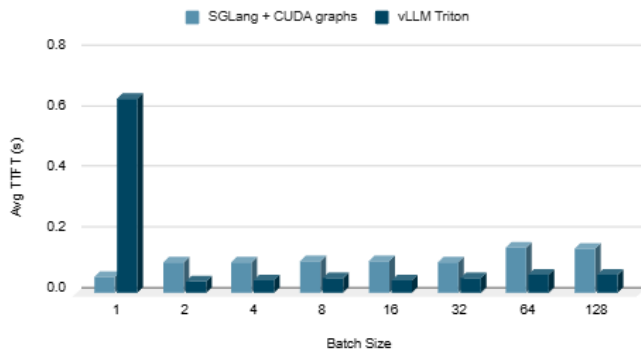


Fig. 8. Batchsize vs TTFT on A100 on GPT-OSS-20B

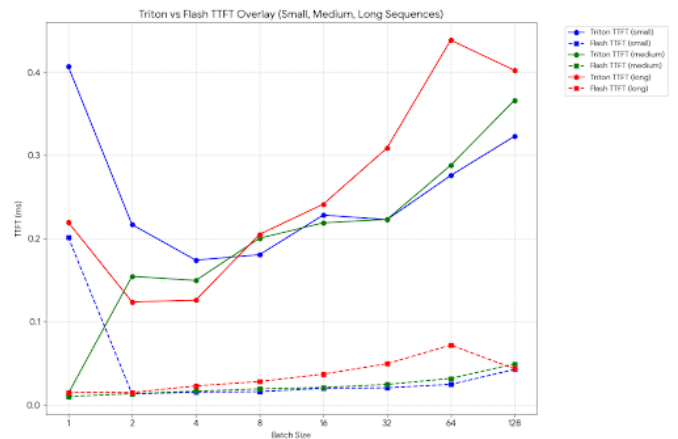


Fig. 11. Average Latency vs Batchsize for Larger Prompts

cation delays between the CPU and the GPU resulting in near-instantaneous response for the first token which is a critical metric for user-facing applications. In contrast, vLLM Triton exhibits a significant latency spike at a batch size of 1. This delay is largely attributed to the standard kernel launch overhead where the system must coordinate multiple individual operations for every request, creating a bottleneck that is only resolved as the workload increases.

As the concurrency increases, the efficiency of the underlying kernels becomes the primary driver of performance. vLLM Flash demonstrates a specific efficiency for medium-scale workloads. Between batch sizes of 2 and 8, the Flash-optimized kernels maintain the lowest average latency and time to first token across all tested configurations. This indicates that while SGLang leads in absolute startup speed for a single user, the vLLM implementation of Flash attention kernels is superior at balancing individual request times once the system transitions out of the single-request state. The transition to high-concurrency processing highlights the strengths of the vLLM Triton backend. Although it starts with the highest initial latency, it achieves the greatest total throughput at the maximum batch size of 64. This suggests that the Triton-based architecture is designed to prioritize raw volume over individual speed. By saturating the GPU with a high number of concurrent tasks, the Triton kernels maximize hardware utilization more effectively than the CUDA graph approach used by SGLang at large scales.

The choice between these two engines represents a fundamental decision between low-latency responsiveness and high-volume output. SGLang with CUDA graph optimizations is uniquely suited for environments where the goal is to provide a single user with the fastest possible interaction. On the other hand, vLLM proves to be a more robust engine for scaling. It manages small groups of requests with high efficiency through its Flash kernels and dominates total token production at high batch sizes. These findings suggest that a unified deployment strategy would require selecting the specific engine version that aligns with the target batch size of the application.

Some of these results were expected given prior knowledge of the underlying implementations and the specific optimizations employed by each framework. In particular, SGLang’s strong performance at Batch 1 and its dominance in Time to First Token aligned well with expectations, as its use of CUDA graphs is widely regarded as a “silver bullet” for minimizing inference latency. Without CUDA graphs, the CPU often becomes a bottleneck, spending more time issuing commands to the GPU than the GPU spends executing them, making SGLang’s low-latency advantage unsurprising. Additionally, the dramatic increase in throughput as batch size grows was entirely anticipated, as modern GPUs are fundamentally optimized for batch parallelism. Observing tokens-per-second scale from roughly 115 at Batch 1 to several thousand by Batch 64 reflects standard hardware utilization behavior rather than an anomaly. It was also expected that differences between frameworks would become less pronounced at larger batch sizes, where kernel efficiency and memory bandwidth

dominate over launch overheads. Finally, the general trend that frameworks optimized for throughput would converge or overtake low-latency-optimized systems at higher batch sizes was consistent with well-established GPU inference principles.

With respect to our GPT-OSS-20B research, our empirical evaluation of GPT-OSS-20B on NVIDIA A100 GPUs identifies a significant performance bottleneck, or “latency cliff,” when operating at a batch size of one. Across varying prompt sizes, we observed that both Time to First Token (TTFT) and total inference latency reach their peak at this minimum concurrency level, likely because fixed kernel launch overheads dominate execution time while the GPU remains under-saturated. The data reveals a dramatic reduction in per-request latency as the batch size scales to two or four, establishing a distinct “efficiency valley” that is critical for real-time serving optimization. These findings suggest that to achieve hardware-efficient deployment, serving configurations must move beyond single-request processing and utilize minimum batching thresholds to effectively amortize initial computational costs.

When evaluating the software backends, vLLM Triton demonstrates superior scaling characteristics for high-concurrency environments compared to SGLang. While SGLang remains competitive at lower scales, vLLM Triton achieves a maximum throughput of approximately 5,500 tokens per second as the batch size reaches 128, making it the preferred choice for high-volume background processing. However, for interactive applications, vLLM Flash provides the most consistent advantage by maintaining the lowest TTFT at small batch sizes. This suggests that while Triton maximizes total hardware utilization, Flash is better suited for user-facing applications where minimizing the perceived delay of the first generated token is the priority.

Ultimately, these results indicate that the most effective deployment strategy involves leveraging dynamic batching to keep the system within the identified “sweet spot” of 4 to 32 concurrent requests. For general-purpose serving, vLLM Triton offers the best balance of total throughput and manageable latency. For specialized low-latency needs, vLLM Flash should be prioritized. Future work should involve testing these configurations under varying prompt lengths, as the efficiency of the FlashAttention kernels used in these engines typically scales more favorably with longer context windows than standard Triton implementations.

VI. LIMITATIONS AND FUTURE WORK

A. Current Limitations

While our benchmarking study provides valuable insights into the performance characteristics of vLLM and SGLang, several limitations constrain the generalizability and depth of our findings.

1) *Single-Instance Deployments*: All experiments were conducted on single virtual machine instances with isolated GPU resources. This approach, while necessary due to budgetary and time constraints, fails to capture the complexity of production-grade inference systems that typically span

multiple nodes. Real-world deployments must handle request routing, load balancing, fault tolerance, and dynamic scaling—none of which were evaluated in our single-node configuration.

2) *Limited Hardware Diversity*: Our testing was primarily confined to NVIDIA A100 GPUs, with minimal data collection on T4 and H100 hardware due to compatibility issues and cluster access limitations. This narrow hardware scope prevents us from making definitive recommendations for cost-sensitive deployments that might utilize mid-range GPUs, or for cutting-edge environments leveraging the latest accelerator architectures.

3) *Constrained Batch Size Range*: The maximum batch size tested was 64 concurrent requests, which represents moderate concurrency for consumer applications but falls short of the hundreds or thousands of simultaneous requests that large-scale production systems must handle. Additionally, we did not explore the behavior of either framework under overload conditions where request queues grow beyond available GPU memory.

4) *Model Size Limitations*: Our primary evaluation focused on TinyLlama (1.1B parameters) with supplementary testing on GPT-OSS-20B. We did not benchmark state-of-the-art models in the 70B+ parameter range, such as LLaMA-2-70B or Mixtral-8x7B, which present fundamentally different memory and compute bottlenecks. The performance characteristics observed with smaller models may not extrapolate to these larger architectures.

5) *Prompt Complexity*: While we utilized the Alpaca dataset to provide realistic instruction-following prompts, we did not systematically vary prompt length, structural complexity, or the presence of multi-turn conversational context. Long-context scenarios (8K+ tokens) and retrieval-augmented generation workflows represent important use cases that were not adequately explored.

B. Future Research Directions

1) *Distributed Inference in Kubernetes*: A critical gap in our current work is the absence of distributed deployment testing. Future research should implement both vLLM and SGLang within a Kubernetes cluster to evaluate their behavior in multi-node configurations. Kubernetes provides native support for horizontal pod autoscaling, rolling updates, and health monitoring—all essential features for production LLM serving.

Specifically, we propose deploying inference workloads across a cluster of GPU-enabled nodes with the following architecture:

- **Ingress Layer**: NGINX or Istio-based load balancer distributing requests across multiple inference pods
- **Inference Pods**: Containerized vLLM/SGLang instances, each with dedicated GPU allocation via Kubernetes device plugins
- **Request Queue**: Redis or RabbitMQ-based message queue to buffer incoming requests during traffic spikes

- **Autoscaling**: Horizontal Pod Autoscaler (HPA) configured to scale pods based on GPU utilization and queue depth
- **Monitoring**: Prometheus and Grafana stack to track per-pod metrics, inter-pod latency, and cluster-wide throughput

This distributed setup would enable investigation of several research questions:

- 1) How does each framework’s performance degrade when requests must traverse network boundaries between pods?
- 2) What is the optimal pod-to-GPU ratio for maximizing cluster utilization while maintaining SLA compliance?
- 3) How effectively can each framework leverage Kubernetes autoscaling to respond to time-varying workloads?
- 4) What are the failure modes when individual pods crash or become unresponsive, and how quickly can Kubernetes recover?

2) *Multi-GPU Parallelism Strategies*: Modern LLM serving increasingly relies on tensor parallelism and pipeline parallelism to distribute model weights and computation across multiple GPUs. Neither framework was tested in multi-GPU configurations in our study, yet this is essential for serving large models that exceed single-GPU memory capacity.

Future work should benchmark:

- **Tensor Parallelism**: Splitting individual transformer layers across multiple GPUs (e.g., 4-way or 8-way splits for 70B models)
- **Pipeline Parallelism**: Assigning different transformer layers to different GPUs in a staged pipeline
- **Hybrid Approaches**: Combining tensor and pipeline parallelism for models requiring 8+ GPUs
- **Communication Overhead**: Measuring the impact of inter-GPU communication via NVLink, PCIe, or InfiniBand on overall latency and throughput

Key metrics to evaluate include strong scaling efficiency (speedup achieved when adding more GPUs to the same workload) and weak scaling efficiency (ability to handle proportionally larger workloads as GPU count increases).

3) *Advanced Stress Testing and Failure Analysis*: Our current benchmarks assume steady-state operation with well-behaved request patterns. Production systems must handle adversarial conditions including:

Traffic Bursts: Sudden spikes in request volume that exceed provisioned capacity. Future tests should simulate realistic traffic patterns using tools like Locust or Apache JMeter to generate bursty workloads with varying inter-arrival times and request durations.

Memory Pressure: Intentionally oversubscribing GPU memory by issuing long-context requests or extremely large batch sizes to observe graceful degradation behavior. Does the framework queue excess requests, reject them with informative errors, or crash?

Fault Injection: Randomly terminating worker processes, simulating GPU errors, or introducing network partitions to

evaluate each framework’s resilience and recovery mechanisms.

Prolonged Load: Running continuous inference workloads for hours or days to detect memory leaks, performance drift, or thermal throttling issues that only manifest over extended operation.

4) *Heterogeneous Hardware Configurations:* Future benchmarks should systematically compare performance across GPU generations and memory capacities:

- Consumer-grade GPUs (RTX 4090, RTX 3090) vs. data center GPUs (A100, H100)
- Impact of GPU memory bandwidth (A100 80GB vs. A100 40GB)
- CPU-GPU interconnect effects (PCIe 3.0 vs. 4.0 vs. 5.0)
- AMD Instinct accelerators as alternatives to NVIDIA hardware

This would provide practitioners with cost-performance tradeoff data essential for infrastructure planning.

5) *Real-World Workload Characterization:* Rather than synthetic benchmarks, future studies should collect and replay traces from production LLM applications to capture realistic request patterns including:

- Multi-turn conversations with session state
- Mixed short and long prompts reflecting actual user behavior
- Retrieval-augmented generation with dynamic context injection
- Function calling and tool use patterns

Partnering with companies deploying LLMs at scale could provide anonymized request logs for this purpose.

6) *Energy Efficiency and Carbon Footprint:* As LLM inference scales to billions of daily requests, energy consumption becomes a critical concern. Future work should measure:

- Joules per token across different frameworks and hardware
- Idle power consumption and efficiency of dynamic batching
- Carbon intensity of inference workloads across different cloud regions
- Tradeoffs between latency targets and energy efficiency

C. Broader System Integration

Finally, LLM inference does not exist in isolation. Future research should evaluate framework performance when integrated with:

- **Caching Layers:** Redis or Memcached for serving repeated queries without GPU invocation
- **Rate Limiting:** Token bucket or leaky bucket algorithms to enforce per-user quotas
- **Model Ensembles:** Routing requests to different model variants based on complexity or urgency
- **Observability Stacks:** OpenTelemetry tracing to track request latency through complex microservice architectures

These integrations are essential for production deployments but add complexity that may interact with framework-specific optimizations in unexpected ways.

VII. CONCLUSION

The rapid adoption of Large Language Models in production environments has created an urgent need for efficient, scalable inference frameworks capable of meeting stringent latency and throughput requirements. This study presents a comprehensive empirical comparison of two leading open-source LLM inference engines—vLLM and SGLang—evaluating their performance across multiple models, batch sizes, and hardware configurations in cloud-based GPU environments.

Our findings reveal that the choice between these frameworks is not a simple matter of one being universally superior, but rather depends critically on the specific deployment context and performance priorities. SGLang, with its CUDA graph optimizations and RadixAttention architecture, demonstrates exceptional performance in low-concurrency scenarios. At batch size 1, SGLang achieves near-instantaneous Time-to-First-Token, making it the optimal choice for interactive applications where initial responsiveness dominates the user experience. This advantage stems from CUDA graphs’ ability to eliminate CPU-GPU communication overhead by replaying pre-recorded operation sequences, effectively removing the kernel launch bottleneck that plagues traditional implementations.

Conversely, vLLM exhibits architectural strengths that become increasingly apparent as concurrency scales. The Flash attention implementation maintains the lowest average latency for medium-scale workloads (batch sizes 2-8), while the Triton backend achieves superior throughput at high concurrency levels (batch size 32-64). These results demonstrate vLLM’s effectiveness at saturating GPU resources when handling multiple simultaneous requests—a critical capability for production systems serving hundreds or thousands of users concurrently. The PagedAttention memory management strategy and continuous batching scheduler underlying vLLM’s design prove particularly effective at maximizing hardware utilization under sustained load.

The cross-pollination of innovations between these frameworks—exemplified by vLLM’s adoption of automatic prefix caching inspired by SGLang’s RadixAttention—highlights the collaborative nature of open-source LLM infrastructure development. Both engines have evolved substantially since their initial research publications, incorporating each other’s core optimizations while maintaining distinct architectural philosophies. This convergence suggests that future performance improvements will likely come from refinements in implementation rather than fundamentally different approaches to KV-cache management and request scheduling.

From a practical deployment perspective, our results support the following recommendations:

For Low-Traffic Applications: Services with sporadic usage patterns, chatbots with few concurrent users, or development/testing environments should favor SGLang. The dramatic reduction in Time-to-First-Token provides immediate user-facing benefits that outweigh throughput considerations when GPUs are underutilized.

For Production-Scale Systems: High-traffic applications requiring consistent performance under load, multi-tenant plat-

forms, or enterprise deployments should select vLLM. Its superior throughput at large batch sizes and robust handling of concurrent requests justify its adoption despite higher initial latency.

For Hybrid Workloads: Organizations serving diverse use cases might benefit from deploying both frameworks in parallel, routing requests based on urgency and batch characteristics. Low-latency critical requests could target SGLang instances, while bulk processing workloads leverage vLLM’s throughput advantages.

Beyond the immediate performance comparisons, this work underscores several broader themes in LLM inference optimization. First, the inherent tension between latency and throughput remains fundamental—optimizations targeting one metric often come at the expense of the other. Second, framework selection cannot be divorced from hardware considerations; the performance characteristics we observed on A100 GPUs may differ substantially on H100s, AMD Instinct accelerators, or future architectures. Third, synthetic benchmarks, while valuable for controlled comparisons, cannot fully capture the complexity of production deployments involving distributed systems, fault tolerance, and dynamic scaling.

The limitations identified in our study—particularly the absence of multi-GPU testing, Kubernetes-based distributed deployments, and comprehensive stress testing—point toward essential future research directions. As LLM inference continues to scale in both model size and request volume, understanding framework behavior in truly production-representative environments becomes increasingly critical. The proposed future work involving Kubernetes orchestration, multi-GPU parallelism strategies, and real-world workload characterization would provide practitioners with the detailed operational guidance necessary for confident infrastructure decisions.

In conclusion, both vLLM and SGLang represent mature, production-ready inference engines with complementary strengths. The decision between them should be driven by careful analysis of expected workload characteristics, performance priorities, and infrastructure constraints rather than assumptions of universal superiority. As the LLM inference landscape continues to evolve rapidly, ongoing empirical evaluation and open sharing of performance data remain essential for advancing the state of the art and enabling practitioners to make informed deployment choices. The frameworks’ continued development, informed by both academic research and production experience, promises further performance improvements that will make large-scale LLM serving increasingly accessible and cost-effective.

ACKNOWLEDGMENT

The authors would like to thank Microsoft Azure for providing the cloud credits that supported the computational requirements of this research. We express our sincere gratitude to NVIDIA and Dr. Wei Bai at NVIDIA for their invaluable mentorship and technical guidance throughout the project. Furthermore, we would like to thank Dr. Alan Liu, Sixian

Xiong, Nengneng Yu, and the entire FROOT Lab for their significant contributions and support throughout the project.

REFERENCES

- [1] W. Kwon et al., “Efficient memory management for large language model serving with PagedAttention,” in *Proc. 29th Symp. Operating Syst. Princ. (SOSP ’23)*, 2023. [Online]. Available: <https://doi.org/10.1145/3600006.3613165>
- [2] Y. Sheng et al., “SGLang: Efficient execution of structured language model programs,” *arXiv preprint arXiv:2312.07104*, 2024. [Online]. Available: <https://arxiv.org/abs/2312.07104>
- [3] K. T. Chitty-Venkata, S. Raskar, and V. Vishwanath, “LLM-Inference-Bench: Inference benchmarking of large language models on AI accelerators,” in *Proc. SC24-W: Workshops Int. Conf. High Perform. Comput., Netw., Storage and Anal.*, 2024.
- [4] J. Chung, S. Lee, and D. Kim, “Bench360: Benchmarking local LLM inference from 360°,” *arXiv preprint arXiv:2511.16682*, 2025.
- [5] AIMultiple Research, “LLM inference engines: vLLM vs LMDe-ploy vs SGLang,” *AIMultiple Research*, 2025. [Online]. Available: <https://research.aimultiple.com/inference-engines/>
- [6] NVIDIA, “LLM inference benchmarking: Fundamental concepts and performance metrics,” *NVIDIA Technical Blog*, 2025.